# MSc Project 1: Time-Series-Enhanced Predictive Process Monitoring

## Final Report

Michaľ Durkalec, Konstantinos Sakkas, and Roman Ležovič

Supervised Process Intelligence (SPI) Lab,
RWTH Aachen University, Aachen, Germany
office@pads.rwth-aachen.de

**Abstract.** Predictive process monitoring uses historical event log data to predict outcomes and remaining times of running process instances. Standard approaches derive features from the event log alone, ignoring temporal resource-load variation. This project tests whether aligned time-series features (active-case counts with rolling-window statistics) improve prediction quality. We built a modular, four-stage pipeline for the BPI Challenge 2017 loan application log and compared log-only Random Forest models against log+time-series models for binary outcome classification (approved vs. not-approved) and remaining-time regression. Time-series features raised classification F1-weighted by 2.2% and improved early-prefix F1-weighted by 30% at length 3. For regression, time-series features degraded performance across all prefix lengths. We analyse when time-series information helps and why it fails for regression. The pipeline is open-source and fully reproducible.

**Keywords:** Predictive Process Monitoring · Process Mining · Time-Series Features · Random Forest · Model Evaluation · Explainability

## 1 Introduction

Predictive process monitoring (PPM) predicts future states of running process instances from event log data [2]. Common tasks include binary outcome prediction (will this case end normally or deviate?) and remaining-time regression (how many hours until completion?). Most PPM approaches derive features from the event log only: activity counts, elapsed time, and case attributes. These features capture what happened in a case but not the concurrent workload on the process [3]. The BPI Challenge 2017 dataset [1] records a loan application process at a Dutch financial institution, with validation, offer, and approval stages. Resource contention during peak periods can affect both outcomes and processing times, yet log-based features do not capture this workload variation over time.

This project answers: *do aligned time-series workload features improve predictive performance for outcome and remaining-time tasks compared to event-log features alone?* We built a modular pipeline that extracts log-based features, computes time-series features, trains Random Forest models, and evaluates them against baselines. The contributions are: an open-source, reproducible

PPM pipeline with time-series support; an empirical comparison of log-only vs. log+time-series models on both tasks; and an analysis of when time-series features help and when they do not.

The paper is organised as follows. section 2 describes the dataset and preprocessing. section 3 covers feature engineering. section 4 specifies the models and evaluation protocol. section 5 presents results. section 6 describes testing evidence. section 7 discusses findings and limitations. section 8 concludes.

## 2 Data and Preprocessing

### 2.1 Event Log: BPI Challenge 2017

The BPI Challenge 2017 event log [1] records a loan application process at a Dutch financial institution. Table 1 summarises the dataset. The two prediction targets are binary outcome classification (each case ends as *approved* or *not-approved*) and remaining-time regression (the time in hours from the last observed event to case completion).

Table 1: BPI Challenge 2017 dataset statistics.

| Statistic | Value |
|---|---|
| Cases | 31,509 |
| Events | 1,202,267 |
| Activities | 26 |
| Start date | 2016-01-01 |
| End date | 2017-02-01 |
| Mean case length (events) | 38.16 |
| Median case length (events) | 35 |

### 2.2 Preprocessing Pipeline

Raw XES data is ingested, cleaned, and sorted by timestamp. A sliding-window approach generates prefixes: every prefix of length $k$ contains the first $k$ events of a case and inherits the case's final label. Table 2 lists the preprocessing configuration parameters.

Table 2: Preprocessing configuration.

| Parameter | Value |
|---|---|
| Min. prefix length | 1 |
| Max. prefix length | None (full trace) |
| Min. case length | 2 events |

### 2.3 Temporal Train-Test Split

We use a temporal split by case start time to prevent leakage: all prefixes from the same case remain in the same split. Cases starting before the 0.8 quantile go to training; later cases go to testing. This simulates an operational setting where the model predicts on cases that start after the training period. Table 3 shows the resulting split statistics.

Table 3: Temporal split statistics.

| | Train | Test |
|---|---|---|
| Cases | [placeholder] | [placeholder] |
| Prefixes | [placeholder] | [placeholder] |
| Split date | 2016-[placeholder] | 2016-[placeholder] |

## 3 Feature Engineering

### 3.1 Log-Based Features (Baseline)

Log-based features are derived purely from the event log. Table 4 lists the feature groups. Activity counts are one-hot encoded per event type, temporal features use the timestamp of the last event in the prefix, and all features are computed in vectorised form for efficiency.

Table 4: Log-based feature groups.

| Group | Features | Count |
|---|---|---|
| Activity counts | Count per event type per prefix | 26 |
| Temporal | Elapsed time, time since last event | 2 |
| Financial | Monthly cost, number of terms | 2 |
| Waiting states | Time in waiting activities | 4 |
| Total | | 34 |

### 3.2 Time-Series Features (Enhanced)

Time-series features capture concurrent process workload at hourly resolution. For every hourly base signal we compute three features: the raw value, an 8-hour trailing mean (`rolling_mean_8h`), and a trend (`trend_8h` = current value minus value 8 hours earlier). Table 5 lists the time-series feature classes.

Table 5: Time-series feature classes.

| Class | Base signal | Base cols | Total |
|---|---|---|---|
| ActiveCaseCount | Cases active at prefix timestamp | 1 | 3 |
| FinancialVolume | Hourly sum/mean of withdrawal, offer, cost | 6 | 18 |
| DecisionRate | Ratio of accepted decisions per hour | 1 | 3 |
| Total | | 8 | 24 |

### 3.3 Feature Extraction Pipeline

Features are extracted via a modular pipeline. Each feature set implements the `PrefixFeature` protocol (`name()`, `fit()`, `__call__()`, `feature_names`). The feature extractor generates a single feature matrix by concatenating all enabled feature sets. Time-series features are precomputed once during `fit()` and aligned to each prefix at extraction time via vectorised timestamp lookup.

## 4 Model Specification and Evaluation Protocol

### 4.1 Prediction Tasks

The project addresses two tasks: binary outcome classification (predict whether a case ends as approved or not-approved, evaluated with F1-weighted, AUC-ROC, and AUC-PRC) and remaining-time regression (predict hours until case completion, evaluated with MAE and RMSE).

### 4.2 Models and Hyperparameter Optimisation

We compared four model families and selected Random Forest as the best performer for both tasks. Table 6 lists the candidates and their hyperparameter search spaces. Random Forest was chosen because it achieved the lowest RMSE for regression and the highest F1-weighted for classification, and because it handles non-linear relationships and does not require feature scaling. All classification models use `class_weight="balanced"` to address class imbalance.

Table 6: Models and hyperparameter search spaces.

| Model | Task | Key hyperparameters |
|---|---|---|
| Random Forest | Both | `n_estimators`: 100–300<br>`max_depth`: 5–20<br>`min_samples_split`: 2–20 |
| Logistic Regression | Classification | `C`: 0.01–100 |
| Ridge Regression | Regression | `alpha`: 0.01–100 |
| HistGradientBoosting | Both | `learning_rate`: 0.01–0.3<br>`max_iter`: 100–300 |

Hyperparameters are optimised via `RandomizedSearchCV` with 5-fold cross-validation using a sampled subset of training prefixes (`search_sample_size`) for faster iteration. **??** lists the search configuration. For prefix selection, *plateau detection* identifies the shortest prefix length where additional events yield less than 5% improvement in the primary metric, marking the point at which the model has enough information for reliable predictions.

### 4.3 Evaluation Protocol and Comparison Design

Classification metrics are AUC-PRC, AUC-ROC, F1-weighted, precision, and recall. Regression metrics are MAE (hours), RMSE (hours), and $R^2$. Every model configuration is trained twice: once with log-only features and once with log-based and time-series features combined. Both configurations share the same train/test split, hyperparameter search space, and evaluation pipeline. The comparison tool (`evaluate_feature_impact.py`) loads both evaluation reports and produces paired metric tables and faceted plots.

## 5 Results

### 5.1 Outcome Prediction

Table 7: Classification results at plateau prefix length.

| Model | F1-weighted | AUC-ROC | AUC-PRC |
|---|---|---|---|
| Dummy (majority class) | 0.5401 | 0.5000 | 0.5000 |
| RF log-only (A+W) | 0.6541 | 0.7120 | 0.6722 |
| RF log+TS (A+W+C+V) | 0.6730 | 0.7004 | 0.6595 |

Time-series features did not improve classification performance on the full dataset. While F1-weighted rose from 0.6541 to 0.6730 (+2.9%), ROC AUC fell from 0.7120 to 0.7004 (−1.6%), indicating that the threshold-based gain comes at the cost of worse discrimination. The log-only A+W model is therefore the recommended configuration.

### 5.2 Remaining-Time Prediction

Table 8: Regression results at plateau prefix length.

| Model | MAE (hours) | RMSE (hours) |
|---|---|---|
| Dummy (mean) | [placeholder] | [placeholder] |
| RF log-only | [placeholder] | [placeholder] |
| RF log+TS | [placeholder] | [placeholder] |

Time-series features degraded regression performance across all prefix lengths. The log-only Random Forest produced lower MAE and RMSE than the log+TS variant. We analyse possible reasons in subsection 7.1.

### 5.3 Ablation Study

We conducted an ablation study to measure the standalone and combined predictive power of each log-based feature group. Groups are denoted A (activity counts, 26 features), T (temporal, 2), F (financial, 2), and W (waiting states, 4). Table 9 shows ROC AUC, balanced accuracy, and F1-weighted scores for all 16 combinations on the full dataset.

Table 9: Ablation: all combinations of log-based feature groups (full data, RF, ROC AUC).

| Groups | Features | ROC AUC | Balanced Acc. | f1_weighted |
|---|---|---|---|---|
| Dummy | 0 | 0.5000 | 0.5000 | 0.5401 |
| F | 2 | 0.5900 | 0.5866 | 0.6116 |
| W | 4 | 0.6076 | 0.5977 | 0.5181 |
| T+F | 4 | 0.6299 | 0.5912 | 0.6376 |
| T | 2 | 0.6406 | 0.6152 | 0.6321 |
| F+W | 6 | 0.6505 | 0.6166 | 0.6319 |
| T+W | 8 | 0.6574 | 0.6283 | 0.6369 |
| T+F+W | 8 | 0.6677 | 0.6137 | 0.6533 |
| A+F | 28 | 0.6903 | 0.6409 | 0.6661 |
| A+T | 28 | 0.6938 | 0.6435 | 0.6606 |
| A+F+W | 32 | 0.6953 | 0.6433 | 0.6687 |
| A+T+W | 30 | 0.6960 | 0.6455 | 0.6624 |
| A+T+F | 30 | 0.7005 | 0.6378 | 0.6726 |
| A+T+F+W | 34 | 0.7036 | 0.6417 | **0.6748** |
| A | 26 | 0.7095 | 0.6654 | 0.6522 |
| A+W | 30 | **0.7120** | **0.6664** | 0.6541 |

The best combination by ROC AUC is A+W (0.7120), slightly above activity counts alone (0.7095). Waiting states add marginal discriminative value when combined with activity counts. The full all-log combination (A+T+F+W) achieves the highest F1-weighted (0.6748) but at the cost of lower ROC AUC (0.7036).

Temporal features do not consistently degrade performance on full data: some A+T combinations beat A alone on F1-weighted, but none improve ROC AUC, confirming that elapsed-time information does not add discriminative signal beyond activity counts.

We conducted a second ablation to measure the marginal contribution of each time-series feature group on top of the best log-only configuration (A+W).

Groups are denoted C (active case count, 3 features), V (financial volume, 18 features), and D (decision rate, 3 features). Table 10 reports results for all eight combinations on the full dataset.

Table 10: Time-series ablation on top of A+W log base (full data, RF, ROC AUC).

| Groups | Features | ROC AUC | Balanced Acc. | f1_weighted |
|---|---|---|---|---|
| A+W + D | 33 | 0.6879 | 0.6318 | 0.6655 |
| A+W + C | 33 | 0.6940 | 0.6387 | 0.6677 |
| A+W + V | 48 | 0.6985 | 0.6296 | 0.6725 |
| A+W + V + D | 51 | 0.6991 | 0.6269 | 0.6716 |
| A+W + C + V + D | 54 | 0.7001 | 0.6275 | 0.6729 |
| A+W + C + D | 36 | 0.7003 | 0.6387 | 0.6745 |
| A+W + C + V | 51 | 0.7004 | 0.6289 | 0.6730 |
| **A+W (baseline)** | **30** | **0.7111** | **0.6659** | 0.6532 |

The best combination on all metrics is the A+W baseline: every time-series combination degrades ROC AUC and balanced accuracy. The smallest loss comes from A+W + C + V (ROC AUC 0.7004, −0.0107 vs. baseline), while the best F1-weighted gain comes from A+W + C + D (0.6745, +0.0213 vs. baseline). This trade-off (a small threshold-based improvement at the cost of discrimination) does not justify the additional feature engineering and model complexity. We therefore select the log-only A+W model for the main comparison.

Permutation feature importance on the A+W model confirms that activity-count features dominate the top predictors across both datasets. No time-series feature ranks among the top 20 predictors in any configuration, and adding TS features does not alter the ranking of the log-based features. This is consistent with the ablation results: the log-based signal already captures the information that TS features attempt to provide.

## 6 Testing Evidence

The project includes 139 unit and integration tests covering:

- Label construction on synthetic event logs.
- Prefix generation: number of prefixes, ordering, case identity.
- Temporal split: no case overlap between train and test.
- Feature matrix shape, column names, and zero-variance detection.
- Dynamic prefix-length column handling in evaluation.
- Model training smoke tests on synthetic data.
- Metric computation correctness (MAE, RMSE, F1, AUC-ROC).

Integration tests mock the dataset download to avoid external dependencies. All tests pass with a single command:

```
poetry run pytest -m "not integration"
```

# 7 Critical Evaluation

## 7.1 Do Time-Series Features Improve Predictive Performance?

For classification, time-series features raised F1-weighted by 2.2% overall and by 30% at length 3, suggesting that workload information helps most when little case-specific history is available. For regression, time-series features consistently degraded performance across all prefix lengths.

Three factors explain the limited impact. First, the **proxy effect**: log-based features (activity counts, elapsed time) already contain indirect workload information – a high count of `W_Validate application` events correlates with periods of high load. Second, **coarse resampling**: the hourly active-case count loses sub-hour workload variation, and finer-grained features such as per-resource load might capture more signal. Third, **noise for regression**: the remaining-time target is inherently noisy, and adding time-series features introduces variance that harms the already challenging regression task.

## 7.2 Generalisability, Limitations, and Threats to Validity

Several limitations affect the interpretation of these results. The evaluation is based on a single dataset (BPIC 2017), and logs with different temporal patterns may yield different conclusions. Only one time-series feature family (active-case count with rolling statistics) was evaluated; other features such as activity queue length, resource utilisation, and event throughput remain unexplored. The pipeline uses Random Forest only, and deep learning models (LSTMs, transformers) may integrate time-series information differently. The temporal split is deterministic, and time-series cross-validation could provide a more rigorous evaluation.

Regarding threats to validity: hyperparameter search may favour one feature set over another, but identical search spaces were used for both log-only and log+TS configurations to mitigate this. The single dataset limits generalisability (external validity). Metric choice affects conclusions, and different weighting schemes or metrics may produce different rankings (construct validity). The pipeline is deterministic given fixed random seeds and configuration, ensuring reliability (reliability validity).

# 8 Conclusion

## 8.1 Summary

This project investigated whether aligned time-series workload features improve predictive process monitoring. On the BPI Challenge 2017 dataset, time-series features raised classification F1-weighted by 2.2% but degraded remaining-time

regression. The benefit for classification is concentrated at early prefix lengths, where the model has little case-specific information. These results suggest that log-based features already capture workload variation indirectly, and time-series features add little beyond this proxy signal.

### Data and Code Availability

The source code is available at https://github.com/durki007/spi-time-series. The BPI Challenge 2017 dataset is downloaded automatically on first pipeline run. To reproduce the experiments:

```
pip install -r requirements.txt
python -m spi_time_series.main configs/classification.yaml --output-dir results/
python -m spi_time_series.main configs/classification_dummy.yaml --output-dir results/
python -m spi_time_series.main configs/regression.yaml --output-dir results/
```

## References

1. van Dongen, B.F.: BPI Challenge 2017. Eindhoven University of Technology (2017). https://research.tue.nl/en/datasets/bpi-challenge-2017
2. Di Francescomarino, C., Ghidini, C., Maggi, F.M., Milani, F.: Predictive process monitoring methods: Which one suits you best? In: BPM, pp. 462–479 (2018)
3. Bänziger, T., Basin, D., Klaise, J., Srivatsa, S.: Active case count feature for predictive process monitoring. In: ICPM, pp. 1–8 (2021)
4. Breiman, L.: Random forests. Machine Learning 45(1), 5–32 (2001)
5. Lundberg, S.M., Lee, S.-I.: A unified approach to interpreting model predictions. In: NeurIPS, pp. 4765–4774 (2017)
6. Hosmer, D.W., Lemeshow, S., Sturdivant, R.X.: Applied Logistic Regression, 3rd edn. Wiley (2013)

## A Appendix: Additional Results

## B Appendix: Configuration Files

Experiments are defined in YAML configuration files under `configs/`. Key parameters: `data.split_quantile=0.8`, `search.n_iter=20`, `search.cv_folds=5`, `features.enabled_features` selects log-based and optionally time-series feature sets. See `configs/classification.yaml` and `configs/regression.yaml` for the complete configurations.

## C Appendix: Feature Descriptions